

1. Smartphone performance optimization:

problem:

Log occurs while switching between apps because frequently used instructions/data may not be available in fast cache memory, causing more accesses to slower DRAM.

Memory hierarchy:-

CPU $\rightarrow$ L1 cache $\rightarrow$ L2 cache $\rightarrow$ L3 cache $\rightarrow$ DRAM $\rightarrow$ storage.

Memory	Speed	Capacity	Role
L1	Fastest	very small	Frequently used instructions core data
L2	very fast	small	Backup for L1
L3	Fast	Larger	Shared cache for CPU cores.
DRAM	Slower	Large	main memory for applic- ations.

Redesign:-

- * Increase L1 cache moderately for critical app data.
- * use a larger L2 cache to reduce accesses to L3.
- * Increases L3 cache for shared application data.
- * use high bandwidth LPDDR5X / LPDDR6 - class DRAM.
- * Apply intelligent cache replacement and prefetching.
- * keep recently used loop app states in memory instead of repeatedly loading them from storage.

Result:-

more cache hits reduce memory access latency, while high bandwidth DRAM improves throughput during multitasking. Therefore, app switching becomes faster and smoother.

2. cloud server memory bottleneck:-

Problem:-

Database queries experience high latency because data may need to travel through multiple memory levels or be fetched from virtual memory/storage.

Recommended design:-

cpu $\rightarrow$ L₁ $\rightarrow$ L₂ $\rightarrow$ L₃ $\rightarrow$ DRAM $\rightarrow$ NVMeSSD.

Cache hierarchy vs virtual memory paging:-

Feature	cache hierarchy	virtual memory paging
Purpose	reduce cpu memory access latency.	Extended available memory.
Typical levels	L ₁ , L ₂ , L ₃	RAM + disk/SSD
Speed	very high	much slower when page faults occur.
unit	cache line	page.
main benefit	Faster repeated data access	Allows large applications.

* Increases L₃ cache for frequently accessed database indexes.

- * use sufficient high-speed DDR5 DRAM.
- * keep database buffer pools in DRAM.
- * use NVME SSD rather than slower storage for paging and persistent data.
- * reduce page faults by allocating enough RAM.

Conclusion:-

Cache, hierarchy should be handle frequently accessed data, while virtual memory should be treated as a backup rather than the primary performance mechanism. more DRAM and efficient caching significantly reduce database latency.

3. Autonomous vehicle data processing:-

An Autonomous vehicle continuously processes camera, LiDAR, radar and other sensor data. therefore, low latency and high band width are critical.

Memory	Advantage	Limitation	Application.
SRAM	Extremely fast, low latency.	Expensive, low capacity	cpu/Gpu cache.
DRAM	high capacity and good bandwidth	slower than SRAM	Sensor buffers, AI data.
MRAM	non-volatile, fast, low stand by power	higher cost/limited capacity	critical persistent data.

proposed hierarchy:

CPU/GPU → SRAM → DRAM → MRAM → Flash/Storage.

- * **SRAM**: Store frequently accessed sensor - processing data and AI working sets.
- * **DRAM**: Store large camera frames, lidar point clouds and intermediate AI sensors.
- * **MRAM**: Store critical configuration, logs and data that must survive power loss.

Result:-

SRAM minimize latency for real-time computation, DRAM provides the required capacity and bandwidth, and MRAM provides the required capacity a fast non-hierarchy supports real-time autonomous decision making.

4. **AI interference 'Edge device':**

The device needs to load AI models quickly while consuming little power.

Memory	Speed	Volatility	Main use.
NAND Flash	moderate / low	non-volatile	permanent model storage.
DRAM	very fast	volatile	Active model execution.
RRAM	Fast, non-volatile	non-volatile	AI model / weight storage.

Proposed hierarchy:-

AI Accelerator/epu → DRAM → RRAM → NAND Flash.

store the complete AI model in NAND flash.

keep frequently used model weights in PRAM.

load active model layers/tensors into DRAM.

use low-power DRAM and power-gating when memory blocks are idle.

Results:-

NAND provides large-capacity persistent storage, PRAM provides fast and low-power non-volatile access and DRAM supports active inference.

5. High performance gaming console:-

problem:-

Large game scenes require large amounts of textures, models and other assets if the CPU/GPU cannot obtain data quickly enough from DRAM, frame rates drop.

L3 cache vs DRAM:-

Feature	L3 cache	DRAM
Latency	very low	higher
capacity	small	large
Bandwidth	high	very high
purpose	Frequently reused data	large game assets.

proposed hierarchy:

cpu/gpu $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ L3 $\rightarrow$ high bandwidth DRAM
 $\downarrow$
NUMESSD.

Enhancements:-

Increase L3 cache to retain frequently reused game data.

Use high bandwidth GDDR6/GDDR6-class memory or newer equivalent for GPU workloads.

Use multiple memory channels to increase bandwidth.